

deep-dive-manuel-administrateur

Manuel d'utilisation — Administrateur Pièces

Version : 2.0 **Date :** Mai 2026 **Plateforme :** Pièces — Marketplace pièces auto d'occasion (Côte d'Ivoire)

À qui s'adresse ce document

Ce manuel décrit l'ensemble des écrans et des leviers dont dispose un administrateur Pièces pour superviser la plateforme : utilisateurs, Liaisons terrain, vendeurs, catalogue, commandes, livraisons, entreprises clientes, paiements, retours, et la nouvelle couche d'audit. Il est tenu à jour à mesure que de nouvelles capacités sont déployées.

Table des matières

1. [Accès et connexion](#)
2. [Architecture rôles et permissions](#)
3. [Tableau de bord et navigation admin](#)
4. [Gestion des utilisateurs \(clients\)](#)
5. [Gestion des Liaisons et journal d'audit](#)
6. [Gestion des vendeurs et scoring](#)
7. [Gestion du catalogue et politique commission](#)
8. [Gestion des entreprises et flottes](#)
9. [Centres de maintenance et plans d'entretien](#)
10. [Stocks tampon entreprise](#)
11. [Gestion des commandes](#)
12. [Gestion des livraisons](#)
13. [Retours et litiges](#)
14. [Paiements et escrow](#)
15. [Notifications et messagerie](#)
16. [Exports CSV et données](#)
17. [Référence API admin](#)
18. [Référence schéma de données](#)
19. [FAQ et dépannage](#)

1. Accès et connexion

Prérequis

- Compte Pièces avec le **rôle ADMIN** attribué dans la table `users.roles`
- Numéro de téléphone ivoirien (+225) enregistré et confirmé

Se connecter

1. Ouvrir `https://pieces.ci` sur navigateur mobile ou desktop.
2. Saisir le numéro de téléphone au format `+225 XX XX XX XX XX` (préfixes acceptés : `01` MTN, `05` Moov, `07` Orange).
3. Cliquer **Envoyer le code** : un OTP à 6 chiffres arrive par SMS.
4. Saisir l'OTP — la vérification est automatique dès le 6e chiffre.
5. Accepter le consentement ARTCI (loi n°2013-450) au premier login.

Accéder à l'espace admin

- URL directe : `https://pieces.ci/admin`
- Lien "Admin" présent dans le header desktop et le menu mobile pour les comptes avec rôle ADMIN
- Le contexte actif (`activeContext`) peut être basculé entre tous les rôles que possède le compte ; pour l'admin, basculer en `ADMIN` (ou `LIAISON` puisque les admins ont aussi ce rôle, voir section 2).

Sécurité

- Authentification 100 % OTP — pas de mot de passe stocké
 - Sessions JWT Supabase avec cookies HttpOnly côté SSR
 - Toutes les routes `/api/v1/admin/*` sont gardées `requireAuth + requireRole('ADMIN')`
-

2. Architecture rôles et permissions

Les 7 rôles de la plateforme

Rôle	Description
MECHANIC	Mécanicien — recherche les pièces, crée les devis tripartites
OWNER	Propriétaire de véhicule — paie les devis, suit les commandes
SELLER	Vendeur de pièces — catalogue, commandes reçues, livraisons
RIDER	Livreur partenaire — courses assignées
ENTERPRISE	Gestionnaire d'une flotte d'entreprise — espace dédié
LIAISON	Employé Pièces terrain — onboarde vendeurs, gère leurs comptes
ADMIN	Administrateur plateforme — supervise tout

Règles d'attribution

Un utilisateur peut cumuler plusieurs rôles. Le champ `activeContext` détermine quel rôle est utilisé pour le contrôle d'accès à chaque requête, mais `requireRole(...)` accepte n'importe quel rôle présent dans le tableau `roles`.

Règle automatique implémentée mai 2026 : chaque fois qu'on attribue le rôle ADMIN à un utilisateur via l'API `PATCH /api/v1/users/:id/roles`, le rôle **LIAISON est automatiquement ajouté** s'il n'y est pas déjà. Logique métier : un admin doit pouvoir faire ce que fait un Liaison sur le terrain quand nécessaire.

Cette règle vit dans `apps/api/src/modules/user/user.service.ts` (`updateRoles`).

Garde côté API

- Routes admin : `preHandler: [requireAuth, requireRole('ADMIN')]`
- Routes Liaison : `preHandler: [requireAuth, requireRole('LIAISON', 'ADMIN')]` — donc un admin peut tout faire ce que fait un Liaison sans bascule de contexte.
- Routes entreprise : `preHandler: [requireAuth, requireRole('ENTERPRISE', 'ADMIN')]`
- Routes vendeur : `requireRole('SELLER', 'ADMIN')`
- Routes livreur : `requireRole('RIDER', 'ADMIN')`

3. Tableau de bord et navigation admin

Tableau de bord (`/admin`)

Page d'accueil de l'espace admin. Affiche les KPI consolidés temps réel via l'endpoint `GET /api/v1/admin/overview` :

Totals :

- Utilisateurs (cumul)
- Vendeurs (cumul)
- Entreprises (cumul)
- Commandes (cumul)
- Commandes actives (status ≠ COMPLETED/CANCELLED)
- GMV en FCFA (Gross Merchandise Value)
- Commissions Pièces cumulées en FCFA

Ce mois :

- Nouvelles commandes
- Nouveaux utilisateurs

Graphique : chiffre d'affaires mensuel (GMV + commissions + nombre de commandes), 12 derniers mois, rendu via Chart.js.

Top vendeurs : classement des 10 meilleurs vendeurs par commissions générées sur les 90 derniers jours.

Navigation latérale (desktop) ou sélecteur (mobile)

Lien	Page	Vue
Tableau de bord	<code>/admin</code>	Overview KPI
Pièces	<code>/admin/parts</code>	Liste catalogue avec filtres
Vendeurs	<code>/admin/vendors</code>	Liste vendeurs paginée + recherche
Clients	<code>/admin/clients</code>	Liste users avec filtres (rôle, recherche)
Entreprises	<code>/admin/entreprises</code>	Liste des entreprises clientes
Liaisons	<code>/admin/liaisons</code>	Liste des Liaisons + audit log (voir §5)
Catalogue (legacy)	<code>/admin/catalog</code>	Vue catalogue v1 (à terme dépréciée)

4. Gestion des utilisateurs (clients)

Liste (`/admin/clients`)

Endpoint: `GET /api/v1/admin/clients/list?q=&role=&page=&limit=`

- **Recherche** : nom, téléphone, email
- **Filtre rôle** : MECHANIC / OWNER / SELLER / RIDER / ENTERPRISE / LIAISON / ADMIN
- **Pagination** : `page` + `limit` (défaut 50, max 200)

Colonnes : nom, téléphone, email, rôles cumulés, nombre de commandes initiées.

Lien direct vers le détail : `/admin/clients/:id` .

Détail (`/admin/clients/:id`)

Endpoint: `GET /api/v1/admin/clients/:id/detail`

Affiche le profil complet, les rôles, le contexte actif, le consentement ARTCI, les commandes passées, les véhicules associés, les avis laissés.

Modifier les rôles

Endpoint: `PATCH /api/v1/users/:id/roles` (body: `{ roles: ["MECHANIC", "ADMIN"] }`)

- Validation via `updateRolesSchema`
- Si ADMIN est ajouté → LIAISON est ajouté automatiquement (cf. §2)
- L' `activeContext` est conservé si encore présent dans la nouvelle liste, sinon le premier rôle

Export CSV clients

Endpoint: `GET /api/v1/admin/export?entity=clients`

Renvoie un fichier CSV horodaté `clients-<timestamp>.csv` avec toutes les colonnes consolidées.

5. Gestion des Liaisons et journal d'audit

Section nouvelle — déployée mai 2026.

Pourquoi

Le Liaison est l'employé Pièces qui démarché les vendeurs sur le terrain, crée leurs comptes, saisit leurs premières pièces et fait agréer la commission. Sans visibilité côté admin, impossible de superviser son activité, détecter les anomalies (sur-commissions, sous-commissions, vendeurs orphelins), ou auditer une transaction litigieuse.

Liste (`/admin/liaisons`)

Endpoint: `GET /api/v1/admin/liaisons`

Affiche tous les utilisateurs ayant le rôle LIAISON dans leur tableau `roles` , avec pour chacun :

Colonne	Source
Nom	<code>users.name</code>
Contact	<code>phone</code> · <code>email</code>
Rôles	<code>users.roles</code> (jointes par virgule)
Vendeurs	Nombre de <code>Vendor</code> où <code>managedByLiaisonId = user.id</code>
Pièces	Nombre de <code>CatalogItem</code> où <code>createdByLiaisonId = user.id</code>
À agréer	Pièces créées par le Liaison avec <code>commissionAcceptedAt IS NULL</code> — affichées en orange si > 0
Actions log	Nombre total d'entrées <code>ActivityLog</code> pour ce Liaison

Cliquer le nom ouvre le détail.

Détail (`/admin/liaisons/:id`)

Endpoint: `GET /api/v1/admin/liaisons/:id`

Trois sections :

Profil

- Nom, téléphone, email
- Rôles cumulés, contexte actif
- Date d'inscription

Vendeurs gérés

- Liste de tous les vendeurs avec `managedByLiaisonId = :id`
- Pour chacun : nom de boutique, contact, commune, statut (PENDING_ACTIVATION / ACTIVE / PAUSED), nombre de pièces, date de création
- Ordre : du plus récent au plus ancien

Pièces saisies (50 dernières)

- Liste des `CatalogItem` où `createdByLiaisonId = :id`
- Colonnes : nom de la pièce, vendeur, prix, commission, **icône agréée** (✓ vert / ⌚ orange), statut catalogue (PUBLISHED/DRAFT/ARCHIVED), date

Journal d'audit

Endpoint: `GET /api/v1/admin/liaisons/:id/activity?page=&limit=`

Affiche en bas de la page détail un tableau chronologique de toutes les actions du Liaison :

Colonne	Contenu
Date	Timestamp ISO formaté en local
Action	Libellé français de l'action (cf. ci-dessous)
Cible	<code>targetType</code> · <code>targetId</code> (8 premiers caractères pour lisibilité)
Détails	Payload JSONB de l'événement

Actions traçables (table `activity_logs`, colonne `action`) :

Code	Libellé	Quand
<code>LIAISON_VENDOR_CREATED</code>	Vendeur créé	POST <code>/liaison/vendors</code>
<code>LIAISON_VENDOR_UPDATED</code>	Vendeur modifié	PATCH <code>/liaison/vendors/:id</code>
<code>LIAISON_PART_CREATED</code>	Pièce ajoutée	POST <code>/liaison/vendors/:id/parts</code>
<code>LIAISON_PART_UPDATED</code>	Pièce modifiée	PATCH <code>/liaison/vendors/:id/parts/:partId</code>
<code>LIAISON_COMMISSION_ACCEPTED</code>	Commission agréée	POST <code>/liaison/vendors/:id/parts/:partId/accept-commission</code>

Pagination : 50 entrées par page par défaut. Les index DB sont sur `(actor_id, created_at DESC)`, `(action, created_at DESC)`, `(target_type, target_id)` et `(created_at DESC)`.

Architecture du log

- Table: `activity_logs`
- Helper: `apps/api/src/lib/activityLog.ts` (`recordActivity()`)
- Best-effort : si la création de log échoue, l'opération métier (création de pièce par ex.) n'échoue pas. C'est volontaire — on ne bloque pas la production sur de la traçabilité.
- Append-only : aucun endpoint admin ne permet la modification ou la suppression d'une entrée.

6. Gestion des vendeurs et scoring

Liste (`/admin/vendors`)

Endpoint: GET `/api/v1/admin/vendors/list?q=&status=&page=&limit=`

Filtres : statut (`PENDING_ACTIVATION`, `ACTIVE`, `PAUSED`), recherche libre (nom de boutique, téléphone, contact, RCCM/CNI).

Colonnes : boutique, contact, type (FORMAL/INFORMAL), KYC, commune, statut, score, nombre de pièces, date de création, Liaison gestionnaire (si présent).

Détail (`/admin/vendors/:id/detail`)

Profil complet + KYC + garanties signées + zones de livraison + catalogue + commandes + reviews + métriques de scoring.

Système de scoring vendeur

Déployé mai 2026. Champs sur le modèle `Vendor` :

Champ	Définition
<code>ordersDelivered</code>	Compteur de commandes confirmées sur 90 jours
<code>disputesOpened</code>	Nombre de litiges ouverts contre ce vendeur sur 90 jours
<code>avgReviewRating</code>	Moyenne des notes laissées par les acheteurs
<code>aggregateRating</code>	Score composite recalculé périodiquement
<code>scoreUpdatedAt</code>	Dernière mise à jour du score

Le score impacte le tri dans les résultats de recherche `/api/v1/browse/search` : les vendeurs mieux notés remontent.

Recalcul du scoring

Pour un vendeur spécifique :

```
POST /api/v1/admin/vendors/:id/recompute-score
```

Pour tous les vendeurs :

```
POST /api/v1/admin/vendors/recompute-scores
```

Opération coûteuse — à lancer hors heures de pointe. Service :

```
apps/api/src/modules/vendor/vendorScore.service.ts
```

Activation d'un vendeur

Un vendeur créé par un Liaison ou via auto-onboarding est en statut `PENDING_ACTIVATION` . Pour le passer en `ACTIVE` :

- Vérifier la signature des deux garanties (`RETURN_48H` et `WARRANTY_30D`) dans `vendor_guarantee_signatures`
- Vérifier que le KYC est rempli (`vendor_kyc`) et que `isPublic = true` si formel
- Update: `UPDATE vendors SET status = 'ACTIVE' WHERE id = '...'`

Une commande ne peut pas être finalisée tant que le vendeur n'est pas ACTIVE.

7. Gestion du catalogue et politique commission

Liste (/admin/parts et /admin/catalog)

Endpoint: GET /api/v1/admin/catalog/list?q=&status=&vendorId=&page=&limit=

Colonnes : photo thumb, nom, catégorie, état (chip coloré), source (OEM/Aftermarket/Compatible), vendeur, prix, commission, statut catalogue, stock, agréée par vendeur, date de création.

Modèle CatalogItem — champs clés

Champ	Type	Rôle
name	String	Libellé de la pièce
category	String	Catégorie libre
oemReference	String	Référence OEM constructeur
vehicleCompatibility	String	Texte libre marque/modèle/année
price	Int (FCFA)	Prix de vente
suggestedPrice	Int (FCFA)	Prix suggéré par l'IA
condition	Enum	NEW / USED / REFURBISHED
partSource	Enum	OEM / AFTERMARKET / COMPATIBLE
warrantyMonths	Int	Garantie vendeur en mois
commissionAmount	Int (FCFA)	Commission Pièces, voir ci-dessous
commissionAcceptedAt	DateTime?	Horodatage de l'agrément vendeur
inStock	Boolean	Disponibilité physique
status	Enum	DRAFT / PUBLISHED / ARCHIVED
aiGenerated	Boolean	Identifié par l'IA
aiConfidence	Float	Score Gemini 0-1
imageOriginalUrl + imageThumbUrl + imageSmallUrl + imageMediumUrl + imageLargeUrl	String	Variantes Sharp
serialPhotoUrl	String?	Photo n° de série / QR
createdByLiaisonId	String?	Lien vers <code>User</code> Liaison
priceAlertFlag	Boolean	Variation > 50 % en < 1h
priceUpdatedAt	DateTime?	Dernier changement de prix

Politique commission

Plancher de sécurité automatiquement appliqué côté serveur :

$commission\ minimum = \max(1000\ FCFA, 5\ \% \times \text{prix de vente})$

Implémentation : `minCommissionFor(price)` dans `packages/shared/validators/catalog.ts` .

Comportement de clamp :

- Quand un Liaison ou un vendeur soumet une commission, le serveur la compare au plancher.
- Si elle est inférieure, le serveur l'enregistre automatiquement au plancher. **Pas de rejet** — c'est volontaire (cf. décision produit mai 2026 : observer les commissions réelles plutôt que les forcer à 5 %).
- Le frontend affiche un message discret quand le clamp va s'appliquer.

Workflow d'agrément :

- Une commission proposée n'est pas engageante. Le vendeur doit explicitement la valider.
- L'agrément se matérialise par `commissionAcceptedAt = NOW()` .
- Pour le Liaison : bouton "Marquer agréée" sur la page édition pièce.
- Pour le vendeur self-service : flag `commissionAccepted: true` dans le PATCH catalog.
- **Si la commission est modifiée**, `commissionAcceptedAt` est remis à `null` automatiquement — un agrément n'est valable que pour le montant exact qui était proposé.

Photos catalog (jusqu'à 3 par pièce)

Modèle `CatalogItemPhoto` :

- `position` : 0 / 1 / 2 (photo principale + 2 secondaires)
- `originalUrl` + variantes redimensionnées
- Endpoints: POST `/catalog/items/:id/photos` , DELETE `/catalog/items/:id/photos/:photoId` , POST `/catalog/items/:id/photos/reorder`

Fitments (catalog_item_fitments)

Ajouté mai 2026. Permet de lier une pièce à des couples *marque / modèle / années / motorisation* structurés (au-delà du `vehicleCompatibility` texte libre).

Modèle `CatalogItemFitment` :

- `brand` , `model` , `yearFrom` , `yearTo` , `engine`
- Un `CatalogItem` peut avoir N fitments.
- Sert à améliorer la précision de la recherche par véhicule (avec décodage VIN).

Références canoniques (ingest)

L'app `apps/ingest` alimente des tables maître (`part_references` , `part_categories` , `vehicle_makes` , `vehicle_models` , `vehicle_generations` , `vehicle_engines`) via :

- **NHTSA**: `pnpm -F ingest start nhtsa` (marques + modèles)
- **NHTSA year enrichment**: `pnpm -F ingest start nhtsa-year`
- **OSM Abidjan**: `pnpm -F ingest start osm` (vendeurs concurrents → `competitor_vendors`)

Ces données alimentent l'autocomplétion vehicle dans les formulaires et le scoring de pertinence dans la recherche.

Détection bait-and-switch

Si le prix d'une pièce PUBLISHED change de plus de 50 % en moins d'une heure, `priceAlertFlag` est mis à `true` et un log warn est émis (`PRICE_ALERT_BAIT_SWITCH`). À surveiller côté admin.

Statut

- **DRAFT** → en cours de création par le vendeur (photos / prix / commission requis pour publier)
- **PUBLISHED** → visible dans la recherche publique
- **ARCHIVED** → masqué mais conservé pour historique

L'archivage se fait via `PATCH /catalog/items/:id` body `{ status: 'ARCHIVED' }`.

8. Gestion des entreprises et flottes

Section nouvelle — feature livrée mai 2026 en plusieurs phases (A→G).

Liste (`/admin/enterprises`)

Endpoint: `GET /api/v1/admin/enterprises/list?q=&page=&limit=`

Colonnes : raison sociale, RCCM, contact principal, nombre de véhicules, nombre de membres, date de création.

Détail (`/admin/enterprises/:id/detail`)

Endpoint: `GET /api/v1/admin/enterprises/:id/detail`

Sections :

- Profil légal (raison sociale, RCCM, RIB, adresse, GPS)
- **Membres** (`enterprise_members`) avec leurs rôles internes (MECHANIC, MANAGER, OWNER, DRIVER, ACCOUNTANT)
- **Flotte** (`vehicles`) avec immatriculation, marque/modèle, année, kilométrage, type d'usage (`vehicleUsageType`), centre de maintenance attitré
- **Commandes** passées au nom de l'entreprise
- **Stocks tampon** (cf. §10)
- **Plans d'entretien** (cf. §9)

Modèle Enterprise — champs clés

Champ	Rôle
<code>name</code>	Raison sociale
<code>rccm</code>	Registre du commerce
<code>rib</code>	Compte bancaire (paiement par virement)
<code>commune</code> + <code>address</code> + <code>lat</code> + <code>lng</code>	Localisation
<code>contactName</code> + <code>contactPhone</code> + <code>contactEmail</code>	Référent
<code>createdAt</code>	Date d'inscription

Espace Enterprise (`/enterprise`)

Côté utilisateur ENTERPRISE :

- `/enterprise/dashboard` : vue d'ensemble flotte (alertes, coûts cumulés)
- `/enterprise/vehicles` : liste de la flotte avec actions par véhicule
- `/enterprise/vehicles/[vehicleId]` : historique maintenance + commandes du véhicule
- `/enterprise/vehicles/import` : import CSV en masse
- `/enterprise/members` : gestion des membres et invitations
- `/enterprise/orders` : commandes consolidées
- `/enterprise/search` : recherche de pièces avec contexte entreprise
- `/enterprise/buffer-stock` : stocks tampon (cf. §10)

Import CSV véhicules

Endpoint: `POST /api/v1/enterprises/:id/vehicles/import` (multipart)

- Format: CSV avec en-têtes (`plate` , `make` , `model` , `year` , `mileage` , `usage_type` , ...)
- Validation: `csvImportRowSchema` (`packages/shared/validators/enterprise.ts`)
- Erreurs ligne par ligne renvoyées en JSON
- Idempotent: doublons par `plate` + `enterpriseId` ignorés

Membres et rôles internes

Modèle `EnterpriseMember` :

- `enterpriseId` + `userId` (unique combo)
- `role` (`EnterpriseMemberRole`): OWNER / MANAGER / MECHANIC / DRIVER / ACCOUNTANT
- `invitedAt` / `joinedAt`

Le mécanisme d'invitation envoie un SMS sur le numéro indiqué ; à la connexion, l'invité accepte et son `User` est rattaché.

9. Centres de maintenance et plans d'entretien

Sections nouvelles — déployées mai 2026 (chantiers 6 et 7).

Centres de maintenance

Modèle `MaintenanceCenter` :

- `name` , `address` , `commune` , `lat` , `lng` , `phone`
- `specialties` : array de catégories prises en charge
- `homeForVehicleCount` : compteur dénormalisé

Vehicle → MaintenanceCenter : chaque véhicule peut avoir un `homeCenterId` (centre de maintenance attribué). Quand une panne survient, on suggère ce centre en priorité.

Endpoints :

- GET `/api/v1/maintenance/centers` (liste publique)
- POST `/api/v1/admin/maintenance/centers` (création par admin)
- PATCH `/api/v1/admin/maintenance/centers/:id`
- POST `/api/v1/entreprises/:eId/vehicles/:vId/home-center` (associer un véhicule à un centre)

Plans d'entretien prédictifs

Modèle `MaintenanceSchedule` :

- `enterpriseId` + `vehicleId` (optionnel — peut s'appliquer à tous les véhicules de l'entreprise)
- `category` (vidange, freinage, etc.)
- `intervalKm` ou `intervalDays`
- `lastDoneAt` / `lastDoneMileage`
- `nextDueAt` / `nextDueMileage`

Recalcul cron : un job tourne quotidiennement et met à jour les `nextDue*` champs en fonction du kilométrage relevé. Les véhicules en alerte apparaissent sur le dashboard entreprise et — pour l'admin — via un endpoint dédié pour audit.

10. Stocks tampon entreprise

Section nouvelle — chantier 9, déployé mai 2026.

Pourquoi

Garantir une disponibilité 24h sur des SKU critiques pour une flotte spécifique (filtres, plaquettes de frein, courroies, huiles). L'entreprise réserve un stock minimum auprès de Pièces ; on engage la disponibilité contre une contrepartie commerciale.

Modèle EnterpriseBufferStock

Champ	Rôle
<code>enterpriseId</code> + <code>catalogItemId</code>	Unique combo
<code>targetQty</code>	Quantité visée
<code>currentQty</code>	Stock effectif (mis à jour à chaque retrait)
<code>autoReplenish</code>	Si <code>true</code> , déclenche commande automatique sous seuil
<code>notes</code>	Texte libre

Statut dérivé (calcul serveur)

Statut	Condition
OUT	<code>currentQty <= 0</code>
LOW	<code>currentQty < targetQty × 0.5</code>
BELOW_TARGET	<code>currentQty < targetQty</code>
OK	<code>currentQty >= targetQty</code>

Endpoints (`/enterprises/:enterpriseId/buffer-stock`)

Méthode	Path	Rôle
GET	<code>/</code>	ENTERPRISE+ Liste
POST	<code>/</code>	OWNER/MANAGER Création
PATCH	<code>/:id</code>	OWNER/MANAGER Modification
POST	<code>/:id/adjust</code>	OWNER/MANAGER/MECHANIC { <code>delta: ±N</code> }
DELETE	<code>/:id</code>	OWNER/MANAGER Suppression

UI (`/enterprise/buffer-stock`)

Tableau avec target vs current, badge de statut coloré, boutons ± 1 pour ajustement rapide, et bannière supérieure comptant les alertes critiques (OUT + LOW). Formulaire de création prend un UUID de `CatalogItem`, target qty, qty initiale, et flag auto-replenish.

11. Gestion des commandes

Liste (`/admin/orders` — endpoint, UI à venir)

Endpoint: `GET /api/v1/admin/orders?status=&page=&limit=`

Colonnes : ID, statut, initiateur (mécanicien), propriétaire, vendeur, total, commission Pièces, date.

Machine à états

Définie dans `apps/api/src/modules/order/order.stateMachine.ts` :

```
DRAFT
  ↓ pay-link generated
PENDING_PAYMENT
  ↓ payment confirmed (CinetPay webhook)
PAID
  ↓ vendor accepts
VENDOR_CONFIRMED
  ↓ rider assigned
DISPATCHED
  ↓ pickup done
IN_TRANSIT
  ↓ delivered
DELIVERED
  ↓ recipient confirms (48h auto)
CONFIRMED
  ↓ payout to vendor
COMPLETED
```

Any non-terminal state can transition to:
CANCELLED (with refund of escrow)

`canTransition(from, to)` est appelé avant toute mise à jour DB pour rejeter les transitions invalides.

Modèle Order — champs clés

Champ	Rôle
<code>shareToken</code>	UUID partagé sur la page <code>/choose/[shareToken]</code> pour le paiement
<code>initiatorId</code>	User mécanicien
<code>ownerId</code>	User propriétaire qui paie
<code>vendorId</code>	Vendeur fournisseur
<code>status</code>	OrderStatus
<code>total</code> (FCFA)	Total dû par l'acheteur
<code>vendorPrice</code>	Prix vendeur reversé
<code>laborCost</code>	Main-d'œuvre mécanicien
<code>deliveryFee</code>	Frais livraison
<code>platformFee</code>	Frais Pièces (TVA incl. si applicable)
<code>paymentMethod</code>	ORANGE_MONEY / MTN_MOMO / WAVE / COD
<code>escrowStatus</code>	HELD / RELEASED / REFUNDED

Modèle OrderItem

- `commissionAmount` figé au moment de la commande (snapshot)
- `vendorPrice` figé
- `condition` et `partSource` snapshot pour traçabilité

Modèle OrderEvent

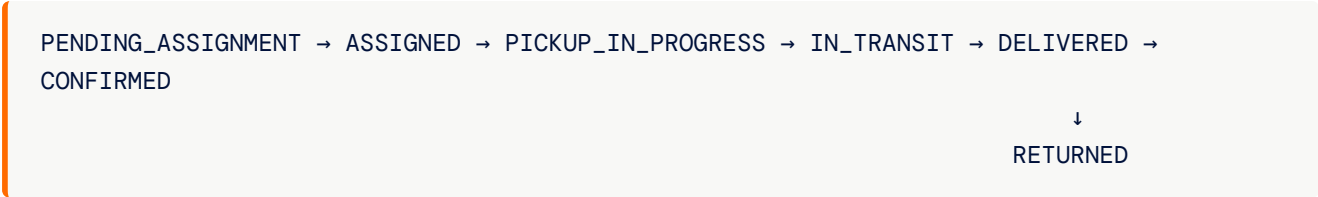
Log d'évènements append-only par commande (états, paiements, livraisons, litiges). Utilisable pour reconstituer l'historique.

12. Gestion des livraisons

Modèle Delivery

Champ	Rôle
orderId	Lien commande
riderId	Livreur assigné
status	DeliveryStatus
pickupAt / pickupAddress / pickupLat / pickupLng	Ramassage chez vendeur
dropoffAt / dropoffAddress / dropoffLat / dropoffLng	Dépose
mode	EXPRESS / STANDARD
codCollected	Espèces collectées (cas COD)

Machine d'états



Endpoints admin

Pas d'écran dédié pour l'instant — supervision via les pages commande et vendeur. Endpoints REST sous `/api/v1/delivery/*` accessibles à l'admin pour debug.

13. Retours et litiges

Litiges (modèle Dispute)

Existait depuis mars 2026. Champs: `orderId` , `openerId` (acheteur), `reason` , `status` (OPEN / IN_REVIEW / RESOLVED_BUYER / RESOLVED_SELLER), `resolutionNote` , `resolvedAt` , `resolvedBy` .
Endpoints: `POST /api/v1/reviews/disputes` (ouverture), `PATCH /api/v1/admin/disputes/:id/resolve` (résolution admin).

Retours structurés (modèle ReturnOrder)

Section nouvelle — chantier 8, déployé mai 2026. Différent du litige : c'est un retour planifié sous garantie ou rétractation 48h.

Champ	Rôle
<code>orderId</code>	Commande d'origine
<code>requesterId</code>	User initiateur
<code>reason</code>	DEFECTIVE / WRONG_PART / CHANGED_MIND
<code>status</code>	REQUESTED / APPROVED / PICKUP_SCHEDULED / RECEIVED / INSPECTED / REFUNDED / REJECTED
<code>refundAmount</code> (FCFA)	Montant à rembourser
<code>inspectionNotes</code>	Diagnostic après réception
<code>createdAt</code> / <code>resolvedAt</code>	

Workflow :

1. Acheteur ouvre un retour (`POST /api/v1/returns`)
2. Admin valide ou rejette (`PATCH /api/v1/admin/returns/:id/decide`)
3. Si validé → pickup planifié, livreur récupère
4. Réception et inspection
5. Remboursement ou rejet final

14. Paiements et escrow

Modèle EscrowTransaction

Champ	Rôle
<code>orderId</code>	
<code>amount</code> (FCFA)	Montant gardé en séquestre
<code>status</code>	HELD / RELEASED / REFUNDED
<code>heldAt</code> / <code>releasedAt</code> / <code>refundedAt</code>	
<code>payoutVendorRef</code>	ID externe du payout au vendeur

Cycle

1. **Order paid** → `EscrowTransaction.status = HELD`
2. **Delivery confirmed by buyer** (manuellement ou auto 48h) → `RELEASED` → payout vendeur déclenché
3. **Litige résolu en faveur de l'acheteur** → `REFUNDED` (remboursement intégral ou partiel selon décision)

Webhook CinetPay

Endpoint: `POST /api/v1/payment/webhook`

- Vérification HMAC signature
- Idempotent: `paymentRef` traité une seule fois
- Met à jour `Order.status = PAID` et crée `EscrowTransaction`

Méthodes de paiement supportées

- ORANGE_MONEY
- MTN_MOMO
- WAVE
- COD (Cash on Delivery — le livreur encaisse)

Configuration

Variables d'environnement à renseigner en prod :

- `CINETPAY_API_KEY`
 - `CINETPAY_SITE_ID`
 - `CINETPAY_SECRET`
 - `CINETPAY_NOTIFY_URL` (webhook public)
-

15. Notifications et messagerie

Préférences utilisateur

Modèle `NotificationPreference` (un par `User`):

- `whatsapp` / `sms` / `email` (booléens)
- `orderUpdates` / `marketing` / `disputes` (granularité)

Endpoint: `PATCH /api/v1/notifications/preferences`

Bot WhatsApp

Module `apps/api/src/modules/whatsapp/` — Meta WhatsApp Cloud API v18.0.

- Webhook: `POST /api/v1/whatsapp/webhook`
- Vérification HMAC SHA-256 (header `X-Hub-Signature-256`)
- Body raw capté par un content type parser scoped au plugin (cf. `CLAUDE.md` — bug connu si on l'enregistre globalement)
- Templates: voir `docs/whatsapp-templates.md`

SMS / Email

Envoi via le module `notification` . Templates dans `apps/api/src/modules/notification/templates/` .

Envoyer une notification manuelle

Endpoint admin : `POST /api/v1/admin/notifications/send`

- Body : `{ userId, channel, template, params }`
- Cas d'usage : escalade litige, relance KYC, communication ad hoc

16. Exports CSV et données

Endpoint unique : `GET /api/v1/admin/export?entity={vendors|clients|orders|catalog}`

- Renvoie un CSV horodaté en `Content-Disposition: attachment`
- Charset UTF-8 avec BOM pour compatibilité Excel français
- Toutes les colonnes consolidées sans filtre (export brut)

Implémentation : `exportCsv()` dans `apps/api/src/modules/admin/admin.service.ts` .

17. Référence API admin

Préfixe : `/api/v1/admin` . Toutes ces routes ont `requireAuth + requireRole('ADMIN')` .

Dashboard

Méthode	Path	Description
GET	<code>/dashboard</code>	Stats de base
GET	<code>/overview</code>	KPI complets + graphique mensuel + top vendeurs

Utilisateurs

Méthode	Path	Description
GET	<code>/users</code>	Liste tous les utilisateurs (paginé)
GET	<code>/clients/list</code>	Recherche/filtre
GET	<code>/clients/:id/detail</code>	Détail client

Vendeurs

Méthode	Path	Description
GET	/vendors	Liste paginée
GET	/vendors/list	Recherche/filtre
GET	/vendors/:id/detail	Détail vendeur
POST	/vendors/:id/recompute-score	Recalcul score
POST	/vendors/recompute-scores	Recalcul global

Catalogue

Méthode	Path	Description
GET	/catalog	Liste sans pagination
GET	/catalog/list	Recherche/filtre

Entreprises

Méthode	Path	Description
GET	/entreprises/list	Liste
GET	/entreprises/:id/detail	Détail
GET	/enterprise/members	Liste des membres (contexte ENTERPRISE)

Liaisons

Méthode	Path	Description
GET	/liaisons	Liste avec stats
GET	/liaisons/:id	Détail (vendeurs + 50 dernières pièces)
GET	/liaisons/:id/activity?page=&limit=	Journal d'audit paginé

Commandes

Méthode	Path	Description
GET	/orders	Liste avec filtre statut

Exports

Méthode	Path	Description
GET	/export?entity=...	CSV

18. Référence schéma de données

Liste exhaustive des modèles Prisma (`packages/shared/prisma/schema.prisma`). Tous mappés en snake_case en DB.

Identité et auth

- `User` — utilisateur + roles
- `DataDeletionRequest` — demandes ARTCI

Activité

- `ActivityLog` — audit append-only (nouveau mai 2026)

Catalogue

- `Vendor` + `VendorKyc` + `VendorGuaranteeSignature`
- `CatalogItem` + `CatalogItemPhoto` + `CatalogItemFitment` (nouveau)
- `SearchSynonym` — synonymes pour la recherche

Commandes

- `Order` + `OrderItem` + `OrderEvent`
- `Delivery`
- `EscrowTransaction`
- `Dispute`
- `ReturnOrder` (nouveau mai 2026)
- `SellerReview` + `DeliveryReview`

Flottes

- `Enterprise` + `EnterpriseMember`
- `Vehicle` + relations vers VehicleMake/Model/Generation/Engine
- `MaintenanceCenter` (nouveau)
- `MaintenanceSchedule` (nouveau)
- `EnterpriseBufferStock` (nouveau)

Référentiels (ingest)

- `VehicleMake` / `VehicleModel` / `VehicleGeneration` / `VehicleEngine`
- `PartReference` / `PartReferenceFitment` / `PartCategory`
- `CompetitorVendor` (OSM)
- `MarketPriceObservation`

Infra

- `Job` — queue async (image variants, AI identify)
- `NotificationPreference`

19. FAQ et dépannage

Q. Un admin n'apparaît pas dans la liste Liaisons. R. Vérifier que `'LIAISON'` est bien dans le tableau `users.roles`. Si ce n'est pas le cas, mettre à jour via `PATCH /api/v1/users/:id/roles` — la logique ajoute automatiquement LIAISON quand ADMIN est dans la liste. Si l'admin existait avant mai 2026, un backfill SQL a été appliqué le 27 mai 2026 :

```
UPDATE users
SET roles = array_append(roles, 'LIAISON'::"Role")
WHERE 'ADMIN' = ANY(roles) AND NOT 'LIAISON' = ANY(roles);
```

Q. Une commission semble bloquée à 1000 FCFA alors que le Liaison a entré 500. R. C'est le plancher de sécurité `max(1000, 5 % × prix)`. Le serveur clamp automatiquement au minimum (cf. §7). C'est volontaire.

Q. La page Liaisons est vide. R. Vérifier qu'il y a bien des utilisateurs avec `'LIAISON'` dans leur tableau `roles`. La table `activity_logs` n'est alimentée qu'après le déploiement du commit du 27 mai 2026 — les actions Liaison antérieures à cette date ne seront pas tracées rétroactivement.

Q. Comment voir TOUTES les actions Liaison sur la plateforme (cross-Liaison) ? R. Endpoint pas encore exposé — il faut requêter directement `activity_logs` :

```
SELECT * FROM activity_logs ORDER BY created_at DESC LIMIT 200;
```

On pourra ajouter `GET /api/v1/admin/activity` si le besoin se confirme.

Q. Un vendeur est en PENDING_ACTIVATION depuis 3 semaines. Pourquoi ? R. Soit le KYC n'a pas été validé, soit les garanties ne sont pas signées (`vendor_guarantee_signatures` doit avoir 2 lignes : RETURN_48H et WARRANTY_30D). Vérifier sur la page détail vendeur (`/admin/vendors/:id/detail`).

Q. Le webhook CinetPay ne marque pas la commande comme PAID. R. Vérifier (1) que l'URL `CINETPAY_NOTIFY_URL` pointe bien vers `https://pieces.ci/api/v1/payment/webhook`, (2) que la signature HMAC est valide, (3) que `paymentRef` n'a pas déjà été traité (idempotence). Logs : `event: PAYMENT_WEBHOOK_RECEIVED`.

Q. Le scoring d'un vendeur n'est pas à jour. R. Le recalcul est cron quotidien. Force immédiat : `POST /api/v1/admin/vendors/:id/recompute-score` .

Q. L'export CSV est en anglais ou les accents sont cassés. R. Le CSV est UTF-8 avec BOM. Excel français doit l'ouvrir correctement ; si problème, Data → Importer données externes → choisir UTF-8.

Q. Une migration n'a pas été appliquée en prod. R. La prod utilise Prisma Postgres (pas Supabase — cf. `db-architecture` mémoire). Render lance `prisma migrate deploy` au démarrage du service `pieces-api` . Si des migrations restent en `not applied` après deploy, ouvrir les logs Render pour voir l'échec, ou appliquer manuellement via `prisma db execute --file ...` puis `prisma migrate resolve --applied <name>` .

Q. Comment ajouter un nouveau type d'action à tracer dans le journal d'audit ? R. (1) Ajouter le code à `ActivityAction` dans `apps/api/src/lib/activityLog.ts` , (2) appeler `recordActivity({...})` dans le handler de route concerné, (3) ajouter le libellé français dans `ACTION_LABELS` de `apps/web/app/(auth)/admin/liaisons/[id]/page.tsx` .

Q. Comment ajouter une page dans la sidebar admin ? R. Éditer `NAV` dans `apps/web/app/(auth)/admin/layout.tsx` . La page est protégée automatiquement par le guard `d'auth/role` du layout.

Annexes

- **Topologie de déploiement** : Web sur Cloudflare Worker (auto-déploi sur push main), API sur Render (auto-déploi sur push, `prisma migrate deploy` au boot), DB applicative sur Prisma Postgres (`db.prisma.io`), DB auth sur Supabase.
- **Source de vérité design** : `DESIGN.md` à la racine du repo.
- **Build des docs PDF/DOCX** : `bash docs/_template/build.sh` (voir `docs/_template/README.md`).

Document interne Pièces — version 2.0 · Mai 2026. À mettre à jour à chaque livraison majeure de nouvelle fonctionnalité admin.